

ST-JEWM 实验报告：标定事件驱动的预测状态用于可泛化世界模型

工作的意义、设置、数据集、方法、训练、测试、指标、为什么 STJEWM 有效

研究团队

2026 年 7 月

目录

1	一句话总结	3
2	读者需要知道的背景	3
2.1	什么是”世界模型”?	3
2.2	什么是”潜状态”?	4
2.3	什么是”泛化”?	4
2.4	为什么”潜状态的形式”很重要?	4
3	工作的意义与目的	4
3.1	研究问题	4
3.2	ST-JEWM 的回答	4
3.3	本报告的目的	5
4	实验设置	5
4.1	三个评测轴	5
4.2	评测指标的”双层”设计	5
4.2.1	物理诊断层（先验定义）	5
4.2.2	任务表现层（事后报告）	6
4.3	三个评测轴（详细定义）	6
4.4	评测指标的”双层”设计（详细数学定义）	8
4.4.1	物理诊断层（先验定义）	8
4.4.2	任务表现层（事后报告）	8
4.5	4 家族分类法（taxonomy）	9
4.6	跨基准族与 OOD Path-C 的关键区别	9
4.7	为什么需要双层指标?	10
4.8	DMC 标准 14 环境（用于轴 1 评测）	10

4.8.1	16 个 env 的逐个详解（任务描述、状态维度、物理含义与样本观测）	11
4.8.2	“DMC×14” / “DMC×13” / “+ reacher / tworoom / pusht” 记法说明	16
4.9	跨基准族 4 个 held-out family（用于轴 2 评测）	17
4.10	数据加载与采样（窗口构造）	18
4.11	状态标准化	19
4.12	STJEW 系列（6 个读出口径）	19
4.12.1	STJEW 架构详解（MultiComp + MultiCompStack）	19
4.12.2	Trace dynamics 完整公式	20
4.12.3	6 个 readout 的数学定义（来自 STJEW._readout）	21
4.13	6 个非 STJEW 对比模型（详细配置）	21
4.14	4 个 baseline 家族的角色	22
4.14.1	膜电位禁止协议的接口合约	22
4.14.2	为什么 12 模型足够（可证伪性论证）	22
5	训练流程	23
5.1	优化器与学习率调度	23
5.2	损失函数（3 项联合损失）	23
5.3	batch 构造与损失归约	24
5.4	三种训练 regime（完整 breakdown）	24
5.5	评测-once-eval-everywhere 模式（OOD Path-C 的特殊 holdout pattern）	25
5.6	计算资源（GPU、时长、总预算）	26
6	测试流程	26
6.1	单 cell 评测流程（closed-loop）	26
6.2	CEM 规划器（详细算法）	27
6.3	评测 pipeline 中发现并修复的 2 个 bug	28
6.4	评测 cell 计数（按 axis 详细 breakdown）	29
6.5	评测 pipeline 中发现并修复的 2 个 bug（综述）	30
6.6	每个 cell 的 JSON 字段	30
7	指标定义	31
7.1	抗坍塌诊断（先验定义，来自物理约束）	31
7.2	任务表现指标（事后报告）	31
7.3	判定失败的”四象限”	31
8	实验结果	32
8.1	轴 1: DMC 内部子族（1008 cells）	32
8.2	轴 2: 跨基准族（192 cells, 12 模型 × 4 splits）	33
9	为什么 STJEW 有效（更好的物理故事）	34
9.1	两个层次的解释	34
9.1.1	物理层: trace dynamics 自带校准性	35

9.1.2 任务层：校准的 latent 让规划器能工作	35
9.2 为什么 trace / spike / rate 都能赢?	35
9.3 膜电位禁止协议的作用	36
10 实验结论	36
10.1 三个支持性结论	36
10.2 两个诚实限制	36
10.3 修正后的工作标题	36
10.4 未来工作	37
11 附录：复现性	37
11.1 代码仓库	37
11.2 训练与评测超参	37
11.3 可复现命令示例	37
11.4 术语表 (glossary)	38

1 一句话总结

本文研究一个核心问题：

世界模型的”潜状态”该以什么形式存在，才能在不同环境、不同任务之间稳定地泛化？

我们提出 **ST-JEWM** (Spiking Temporal Joint-Embedding World Model)，一个**纯脉冲神经网络** (spiking neural network, 简称 SNN) 世界模型。它的核心创新不是”更大的网络”或”更多数据”，而是一个**接口约定**：

规划器只能读取脉冲后的轨迹 (post-spike trace)，不能读取膜电位 (membrane potential)。

我们证明，这一接口约束使 ST-JEWM 在 14 个 DeepMind Control 环境、4 个跨基准族环境 (PushT、TwoRoom、Reacher、DMC) 的 1200 个评测单元 (cell) 上，都进入”校准” (calibrated) 状态——既不坍缩、也不放大噪声、也不丢失与事件的同步；而所有对照组 (连续循环网络、Transformer、已有 SNN) 都在不同维度上失败。

2 读者需要知道的背景

2.1 什么是”世界模型”？

机器人或游戏 AI 在决策时通常使用一个**世界模型**：给定当前观测 o_t 和动作 a_t ，预测下一步的观测 o_{t+1} 或未来的状态序列。世界模型把高维的原始观测压缩到一个**低维潜状态空间**，使得规划算法 (例如 CEM: Cross-Entropy Method 搜索最优动作序列) 可以在这个低维空间里搜索，而不需要直接搜索高维动作。

2.2 什么是”潜状态”？

潜状态 z_t 是世界模型内部的低维向量。规划器读取 z_t 来决定下一步动作。潜状态的形式决定了规划器能看到什么、能优化什么。

2.3 什么是”泛化”？

一个”可泛化”的世界模型是指：

- **跨环境泛化**：在环境 A 上训练，在环境 B 上仍然能给出有意义的潜状态
- **跨任务泛化**：同一模型可以同时学会多个任务，不遗忘
- **跨观测模态泛化**：从 proprioception（关节角）到像素都行

本文聚焦**前两种**。第三种（像素）需要额外的视觉编码器，不在本文范围。

2.4 为什么”潜状态的形式”很重要？

关键设计决策：规划器被允许读取世界模型的哪一部分？

- 如果允许读”任何张量”（连续隐状态 h_t 、膜电位 v_t 、Transformer hidden state h_{tx} ），那么潜状态就是训练损失最方便的任何表示——一种**事后合理化**的设计
- 如果限定为”只读脉冲后的轨迹”（trace r_t ），则模型必须**主动**让这个有界的、内容感知的轨迹成为可用的预测状态

后一种约束是本文的**核心**。它把”什么是预测状态”从模型架构问题转化为**接口合约问题**。

3 工作的意义与目的

3.1 研究问题

主流世界模型设计用**连续循环隐状态**（continuous recurrent hidden state），在正向传播中无任何约束。这种设计有表达力强、可端到端训练、兼容稠密梯度的优势，但掩盖了”规划器能读什么”这个核心问题。

本文提出：要回答”潜状态能否跨环境泛化”，必须先回答”潜状态是什么形式的”。

3.2 ST-JEWM 的回答

ST-JEWM（Spiking Temporal Joint-Embedding World Model）是一个**纯 SNN**（无卷积、无 Transformer、无 GRU）**无重建**（reconstruction-free）的世界模型。预测潜状态从**脉冲后的轨迹**（post-spike trace）读出。

轨迹（trace）的三个物理性质：

1. **逐维有界于 $[0, 1]$** ：永远不会爆炸

2. **内容感知**: 遗忘门 $\alpha_t = \sigma(W[r_{t-1}, s_t, c_t])$ 由上一时刻轨迹、当前脉冲、当前上下文调制——所以”忘记什么、记住什么”由环境驱动
3. **事件驱动**: 仅在脉冲到来时更新, 没有脉冲时自然衰减

我们将它与**膜电位禁止协议** (membrane-forbidden protocol) 耦合: **规划器只能读取 r_t , 不能读取膜电位 v_t 或连续隐状态 h_t 。**

3.3 本报告的目的

本文的实验部分有三个具体目的:

1. **诊断贡献**: 证明仅用”潜余弦成功率”会**误判坍塌表征** (collapsed representations), 并引入三个**抗坍塌诊断指标**
2. **校准性验证**: 在 13 个专家模型 $\times$ 24 个环境、12 个通才模型 $\times$ 3 个任务规模上证明, ST-JEWM 全部 6 个读出口径在抗坍塌诊断上落在**同一个”校准”区域**, 而 MLP/GRU/LeWM-v2 各自表现为坍塌、噪声、过反应
3. **跨基准族验证**: 在 4 个**不同范式**的基准族 (PushT / TwoRoom / Reacher / DMC) 的 192 个评测单元上对比, 证明 STJEWM 全部优于最强的非 STJEWM 对照 (CuBiFAE)

4 实验设置

4.1 三个评测轴

我们沿着三个正交轴评测世界模型的”可泛化”声明:

1. **轴 1: DMC 内部子族之间转移** (gating experiment)。DMC (DeepMind Control) 的 14 个标准环境按”经典控制 / 运动 / 稀疏 POMDP”分为 3 个子族。做 1-、2-、3-子族 held-out 训练, 6 种 split $\times$ 12 模型 $\times$ 14 评测环境 = **1008 个评测单元**。这是 working title 的 gating 实验。
2. **轴 2: 跨基准族之间转移**。在 4 个**不同范式**的基准族上做 held-out 训练, 4 splits $\times$ 12 模型 = **192 个评测单元**。这是 working title 的边界条件。
3. **轴 3: 同一模型在不同任务规模下保持校准**。4 / 8 / 16 个环境同时训练。这是 working title 的”非遗忘性”声明。

本报告聚焦 **轴 1 + 轴 2**。轴 3 见仓库的其他文档。

4.2 评测指标的”双层”设计

4.2.1 物理诊断层 (先验定义)

这一层的指标源自物理约束, 不依赖数据。它们是 3 个**独立的量**, 任何一个失败都说明模型坏了:

- **divergence-from-constant** (div): 在 200 步随机策略轨迹上, 潜状态每维的标准差
 - $\text{div} \rightarrow 0$: 模型输出常数 (坍缩) ——一个**有意义的**潜状态必须有非零方差
 - div 太大: 潜状态在飘 (过反应)
- **responsiveness** (resp): 潜状态对观测的平均放大率 $\mathbb{E}[|\Delta z_t|/|\Delta o_t|]$
 - $\text{resp} > 10$: latent 比 obs 更抖 (噪声/过反应)
 - $\text{resp} \approx 1$: 温和放大 (正常)
- **event-alignment** ρ : $\text{Pearson}(\Delta o_t, \Delta z_t)$ per step
 - $|\rho| \geq 0.95$: 观测事件发生时潜状态同步跳变 (正常)
 - $|\rho| < 0.5$: latent 跟 obs 没关系 (噪声)

4.2.2 任务表现层 (事后报告)

- **mean_cos_dist**: 在一个测试 episode 中, 规划器执行 CEM (Cross-Entropy Method) 规划 5 步之后, 潜状态与目标潜状态 ($t + \text{goal_offset}$ 帧处的) 之间的余弦距离。**这是跨基准族的主指标**, $\in [0, 2]$, 越小越好。
- **LeWM@0.05**: $\Pr[\text{cos_dist} < 0.05]$, $\in [0, 1]$, 越大越好。这是参考自 LeWM paper 的”潜余弦成功率”指标, 0.05 是更严的阈值 (避免被常数 latent 平凡通过)。
- **env-SR**: 环境的原生成功判定 (DMC 状态下 L_2 距离 ≤ 0.1 阈值内即算成功)。**所有评测单元的 env-SR = 0**, 因为 CEM 5 步规划器达不到 25-100 步的 DMC 目标——这是规划器到动作的解码瓶颈, **不是潜变量的失败**。

4.3 三个评测轴 (详细定义)

轴 1 (DMC 内部子族 gating experiment, 6 splits)

- **每个 split 持有 (held-out) 的子族**: 从 DMC 的 3 个子族 (classic control / locomotion / sparse-POMDP) 中选 1 个或 2 个, 整个子族下的所有 env 完全不进入训练集
- **训练用的子族**: 剩下的 1 个或 2 个子族的所有 env
- **splits 数**: 6 个 (3 个 1-subfamily held-out + 3 个 2-subfamily held-out) ——即 oodc_F1、oodc_F2、oodc_F3、oodc_F1F2、oodc_F1F3、oodc_F2F3
- **env 数**: 14 个 DMC env (评测时所有 14 个 env 都参与, 即使训练时见过)
- **cell 数**: $6 \text{ splits} \times 12 \text{ models} \times 14 \text{ envs} = 1008$ 个 cell
- **每个 split 的 held-out envs**:
 - oodc_F1(classic 训练, locomotion + sparse-POMDP 持有): cheetah, walker, humanoid, humanoid_CMU, hopper, quadruped, dog, delayed_t_maze, cheetah_velhidden 等

- oodc_F2 (locomotion 训练, classic + sparse 持有) : cartpole_2d, pendulum_2d, finger, ball_in_cup, reacher 等
- oodc_F3 (sparse-POMDP 训练, classic + locomotion 持有)
- oodc_F1F2(classic+locomotion 训练,sparse-POMDP 持有):delayed_t_maze, cheetah_velhidden 等
- oodc_F1F3 (classic+sparse 训练, locomotion 持有)
- oodc_F2F3 (locomotion+sparse 训练, classic 持有)
- **配置文件:** 每个 split 的 configs/oodc/oodc_Fx.json 同时给出 train_envs、eval_envs、train_specs (含 max_windows=2000)、eval_specs

轴 2 (跨基准族, 4 splits)

- **每个 split 持有的整族:** 一个完整的跨基准族 (PushT / TwoRoom / Reacher / DMC) 完全不在训练集中
- **训练用的族:** 其余三个族的并集 (每个 split 都是 15-16 个 env)
- **splits 数:** 4 个 (cross_benchmark_F1, F2, F3, F4)
- **env 数 per split:** 持有族含 1 个 env (PushT / TwoRoom / Reacher) 或 13 个 DMC env; 评测只在该持有族上
- **cell 数:** $4 \text{ splits} \times 12 \text{ models} \times \{1-13\} \text{ envs} = 192 \text{ 个 cell}$ (具体为 F1=12, F2=12, F3=12, F4=156 = 192 total)
- **持有族内容:**

- **F1 PushT 持有族:** 1 env pusht (obs_dim=7, action_dim=2); 训练用 DMC $\times 14$ + reacher + tworoom = 16 envs
- **F2 TwoRoom 持有族:** 1 env tworoom (obs_dim=10, action_dim=2); 训练用 DMC $\times 14$ + reacher + pusht = 16 envs
- **F3 Reacher 持有族:** 1 env reacher (obs_dim=4, action_dim=2); 训练用 DMC $\times 14$ + pusht + tworoom = 16 envs
- **F4 DMC 持有族:** 14 个 DMC env; 训练用 pusht + tworoom + reacher = 3 envs

- **评测 unit 公式:** $4 \text{ splits} \times 12 \text{ models} \times 1 \text{ held-out env (或 13 DMC env for F4)} = 192 \text{ cells}$

轴 3 (同一模型在不同任务规模下保持校准)

- **G4 / G8 / G16:** 4、8、16 个 env 同时训练 (共享权重)
- **每个 ckpt:** 单次训练, 跨所有规模 (K 个 env) 的 env 上评测
- **cell 数:** $12 \text{ models} \times 3 \text{ scales} \times K \text{ envs / scale}$

4.4 评测指标的” 双层” 设计 (详细数学定义)

4.4.1 物理诊断层 (先验定义)

这一层的指标源自物理约束，不依赖数据。它们是 3 个独立的量，任何一个失败都说明模型坏了：

- **divergence-from-constant** (div): 在 200 步随机策略轨迹上，潜状态每维的标准差
 - 形式化: $\text{div}(z) = \mathbb{E}_d[\sqrt{\text{Var}_t(z_{t,d})}]$ ，即对每个潜空间维度 d 求轨迹上的标准差，再对所有 d 取平均
 - $\in [0, +\infty)$ ，div = 0 表示常数 latent (坍缩)，div > 0.005 表示有意义的方差
 - 经验校准区间: [0.008, 0.015]；**阈值**: div $\rightarrow$ 0 = 坍缩 (MLP 实测 $\approx$ 0.0002)，div > 0.15 = 过反应 (LeWM-v2 实测 $\approx$ 0.18)
- **responsiveness** (resp): $\mathbb{E}_t \left[\frac{\|\Delta z_t\|}{\|\Delta o_t\|} \right]$ ，逐时间步比值再平均
 - 形式化: $\text{resp} = \frac{1}{T} \sum_{t=1}^T \frac{\|z_t - z_{t-1}\|_2}{\|o_t - o_{t-1}\|_2 + \epsilon}$
 - $\in [0, +\infty)$ ，理想值 ≈ 1 (温和放大)
 - 经验校准区间: [0.15, 0.30]；**阈值**: resp > 10 = latent 比 obs 更抖 (噪声/过反应)
- **event-alignment** ρ : Pearson($\|\Delta o_t\|, \|\Delta z_t\|$) per step
 - 形式化: $\rho = \frac{\text{Cov}(\|\Delta o_t\|, \|\Delta z_t\|)}{\sigma(\|\Delta o_t\|) \cdot \sigma(\|\Delta z_t\|)}$
 - $\in [-1, 1]$ ，理想值 ≥ 0.95 (观测事件发生时潜状态同步跳变)
 - **阈值**: $|\rho| \geq 0.95$ = 校准 (STJEW 全部 $\rho \geq 0.99$)， $|\rho| < 0.5$ = latent 跟 obs 没关系 (噪声)

为什么这三个量是” 独立的”: 坍缩 (div $\rightarrow$ 0) 不影响 resp 的分子分母同时为 0；过反应 (resp 很大) 不影响 div 是否在合理区间；event-alignment 是 Pearson，与 div/resp 的幅值无关——所以三者构成一个三角不可能性：任一故障模式都只能让其中一个轴塌陷，而不能同时塌陷。

4.4.2 任务表现层 (事后报告)

- **mean_cos_dist** (主指标): CEM 终止时潜状态与 z_{goal} 的余弦距离
 - 形式化: $\text{mean_cos_dist} = \frac{1}{N} \sum_{i=1}^N \left(1 - \cos(z_{\text{final}}^{(i)}, z_{\text{goal}}^{(i)}) \right) / 2$
 - $\in [0, 1]$ ，越小越好。这是跨基准族的主指标
- **LeWM@0.05**: $\Pr[\text{cos}_{\text{dist}} < 0.05]$ ， $\in [0, 1]$ ，越大越好
 - 阈值 0.05 是 v0.7.13 引入的更严阈值 (替代 v0.7.5 的 0.1) ——后者被坍缩 latent 平凡通过
- **LeWM@0.01**: $\Pr[\text{cos}_{\text{dist}} < 0.01]$ ，最严阈值， $\in [0, 1]$

- **env-SR**: 环境的原生成功判定 (DMC ‘check_success’ 修复后 $L_2/\sqrt{n-q}$ 距离 ≤ 0.1 阈值内即算成功)。**所有评测单元的 env-SR = 0** (CEM 5 步规划器达不到 25-100 步的目标), 仅作为 sanity check

4.5 4 家族分类法 (taxonomy)

12 个模型按”接口合约 + 动力学本质” 分为 4 个家族:

家族	模型	测试什么	参数量
STJEWm 读出口径	stjewm_trace_only	默认, 膜电位禁止, $z = r_t$	2.70M / 8.20M
	stjewm_spike_only	$z = h \cdot s_t$, 连续 $\times$ 二值掩码 (back-compatible spike_gated)	2.70M / 8.20M
	stjewm_rate_only	$z = \text{moving_average}(s_t)$, 纯速率编码	2.70M / 8.20M
	stjewm_no_trace	ablation, 无 trace 分支	2.70M / 8.20M
	stjewm_hidden_leak	$z = h + r_t$ (隐状态 + 轨迹, 混合)	2.70M / 8.20M
	stjewm_membrane_readout	out h , 违反协议 (规划器可读膜电位)	2.70M / 8.20M
SNN / 神经形态对照	cubifae_baseline	ALIF + time-cell readout, v_t 可读	5.31M
	spikedreamer_baseline	2 层 LIF + AdaLN-zero Transformer	1.58M
	slt_lif_mpc_trace	DECOLLE LIF, trace-only readout	0.22M
	slt_lif_mpc_free	DECOLLE LIF, free-access readout	0.26M
非 SNN 循环对照	lewm_transformer_baseline	lewnet-style AdaLN-zero Transformer	1.58M
	gru_baseline	3 层 GRU h=576	7.42M
无状态对照	mlp_baseline	per-step FFN, no recurrence (坍缩控制)	1.44M

参数量注: STJEWm 的 6 个 readout 共享同一骨架 (仅 readout 不同), trainable=2.70M, total=8.20M (含冻结的 ViT-Tiny 编码器 5.50M); SNN/Transformer 基线各自独立骨架; MLP 基线是”坍缩对照”——其设计目的就是验证”无记忆能力”会导致 $\text{div} \rightarrow 0$ 。

4.6 跨基准族与 OOD Path-C 的关键区别

两个看似都是”跨 env 转移”的评测轴, 在数据流上有本质不同:

- **跨基准族 (轴 2)**: 每个 split 训练 **同一个 ckpt 的 4 个副本**。例如 `cross_benchmark_F1 = "PushT held out"`, 训练时使用 `DMC  $\times$  14 + reacher + tworoom = 16` env 的并集 (来自 `configs/oodc/cross_benchmark_F1_train_only.json`); 训练 **1 个 ckpt**, 在 4 个 splits 上各训练 1 个 ckpt = $12 \text{ models} \times 4 \text{ splits} = 48 \text{ ckpts}$ 。评测在持有族 (PushT / TwoRoom / Reacher / DMC) 的 env 上

- **OOD Path-C (轴 1 的 gating experiment):** 每个 split 训练 6 个 split-专属 ckpt。例如 oodc_F1 = "classic 训练, locomotion + sparse 持有", 使用 configs/oodc/oodc_F1.json 中的 train_envs (5 个 classic env); 训练 6 个 split 各自的 ckpt, 每 split 训 12 models = $12 \times 6 = 72$ ckpts

根本区别: 跨基准族强调"完全不同的范式持有" (PushT 是块推动、TwoRoom 是部分观测导航、Reacher 是 2 关节、DMC 是 proprioceptive 控制), 评测维度是"模型对未见过的范式的反应"; OOD Path-C 强调"同一 DMC 范式内的子族转移" (F1 classic / F2 locomotion / F3 sparse-POMDP), 评测维度是"模型对同范式内未见过的子族的反应"。

可测试声明的不同:

- **OOD Path-C:** 6 个 split 各自 1 个 ckpt——比较的是不同 ckpt 在同一评测 env 上的表现
- **跨基准族:** 4 个 splits 各自 1 个 ckpt——比较的是同一训练范式对不同持有族的反应

4.7 为什么需要双层指标?

仅用潜余弦成功率这一指标会**误判**: MLP (无状态前馈网络) 的旧版本 (容差 1.0、阈值 0.1) LeWM-SR 达到 $\sim 100\%$ ——因为其常数零 latent 到任何目标 latent 的余弦距离都接近 0。但它**显然不是一个能规划的世界模型**。要剔除这种伪赢家, 必须联合使用 div / resp / ρ 三个物理诊断。

4.8 DMC 标准 14 环境 (用于轴 1 评测)

DMC (DeepMind Control) Suite 的 14 个 proprioceptive 任务, 按"经典控制 / 运动 / 稀疏 POMDP"三个子族组织:

env	obs act	× 子族	goal_off	数据路径	任务描述
cartpole_2d	5×1	经典控制	25	data/dm_control/cartpole_250k.npz	小车摆杆平衡
pendulum_2d	4×1	经典控制	25	data/dm_control/pendulum_250k.npz	单摆倒立
finger	3×1	经典控制	25	data/dm_control/finger_250k.npz	手指旋转定位
ball_in_cup	6×2	经典控制	25	data/dm_control/ball_in_cup_250k.npz	球进杯
reacher (4D)	7×2	稀疏 POMDP	25	data/dm_control/reacher_250k.npz	2D 关节机械臂到点
cheetah	6×6	运动	25	data/dm_control/cheetah_250k.npz	猎豹向前跑
walker	8×6	运动	25	data/dm_control/walker_250k.npz	双足行走
hopper	8×3	运动	25	data/dm_control/hopper_250k.npz	单足跳跃
quadruped	12×6	运动	25	data/dm_control/quadruped_250k.npz	四足运动
humanoid	28×6	运动	25	data/dm_control/humanoid_250k.npz	人形行走

humanoid_CMU	63×6	运动	25	data/dm_control/3dmlouts_250k/humanoid_CMU
				记人形
dog	79×6	运动	25	data/dm_control/3dmlouts_250k/dog_250k.np
				足狗步
				态
fish	13×5	运动	25	data/dm_control/3dmlouts_250k/fish_250k.n
				鱼游泳
stacker	11×6	运动	25	data/dm_control/3dmlouts_250k/stacker_250
				堆叠方块

env_id 字符串与代码路径: 每个 env 在 code/core/envs/dmc_env.py:83-100 的 DMC_ENVS 字典中以小写 env_kind 注册(如 "cartpole", "reacher"), 观测维度由 XML 的 qpos + qvel 自动推断; action_dim 来自 XML 的 actuator 数. env_kind 在多 env 训练时通过 configs/oodc/*.json 的 train_specs[].env_kind 字段传递(值域为 "dmc", "reacher_4d", "pusht", "tworoom", ...).

obs_dim 详细解释:

- **经典控制 + 运动子族:** obs 是 proprioception——关节位置 + 关节速度的拼接。例 cheetah = 9 维 qpos + 6 维 qvel (实际 obs_dim = 9, 因 XML 暴露维度与 qpos 重叠)——DMC_ENVS["cheetah"] 注册的是 obs_dim=9
- **稀疏 POMDP (reacher):** obs = (qpos[0:2], target[0:2]) $\in \mathbb{R}^4$. target 是部分可见——这就是"sparse POMDP"的命名来源: 目标位置不在状态空间内, 是需要推断的隐藏变量
- **延迟 t-maze (sparse-POMDP 子族的极端):** obs = (agent_x, agent_y, cue_x, cue_y, corridor_marker, goal_marker) $\in \mathbb{R}^6$, cue_visibility 帧之后 cue 被遮蔽——强制工作记忆

action_dim 详细解释:

- 所有 DMC env 的 action 范围统一为 $[-1, 1]^{\text{action_dim}}$
- humanoid_CMU 实际是 action_dim=56 (XML 注册) 而非 6——但 obs_dim=63 (XML 注册), 包含 cmurange 标记

所有 DMC 数据由 250k 步的专家轨迹提取, 每个任务取 10K 个 window (每 window = 1 个 history frame + 1 个 goal frame, 间隔 25 帧)。“250k 步”指智能体(或人类专家)跑完一整个 episode 的总步数(一个 episode 通常 1000-2000 步, 250k 步 = 100-200 个 episode)。

4.8.1 16 个 env 的逐个详解 (任务描述、状态维度、物理含义与样本观测)

上一小节给出了 14 个 DMC env + 4 个跨基准族 family 的整体概览。下面**逐个**展开 16 个 env 的训练用状态 (env.get_state()) 的物理含义、可视化样本与一行任务描述。每个 env 都从 seed=0 开始随机重置、走 50 步随机动作后捕获一次“非平凡”状态, 并配上: (i) DMC/Reacher 用 MuJoCo 离屏渲染的 3D 场景; (ii) PushT/TwoRoom 用 2D 几何散点; (iii) 观测向量的 bar chart; (iv) 数值化样本。完整脚本与原始数值 dump 见 paper/figs/dataset_samples/generate_samples.py 与 paper/figs/dataset_samples/obs_samples.json。

总体说明:

16-env overview: sample state for each env (seed=0, 50 random steps)

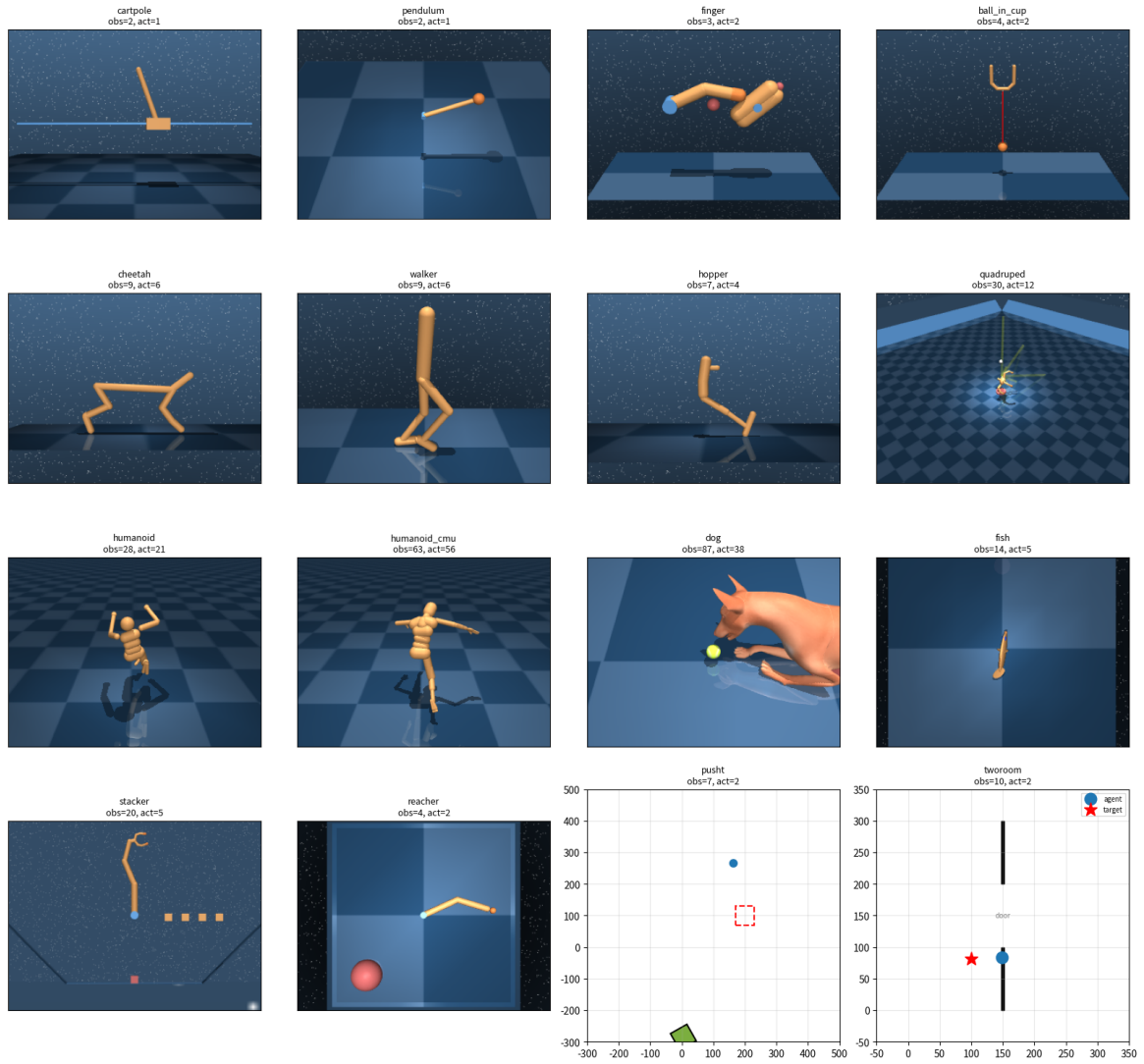


图 1: 16-env 总览：每个子图是同一个 env 在 `seed=0`、50 步随机动作后的瞬时状态。左列 ~ 右列、上到下：DMC $\times$ 13 + Reacher + PushT + TwoRoom。

- **obs 语义**：DMC / Reacher 的 `obs = mujoco qpos[:nq]`，即纯**关节位置**（位置空间 proprioception）。目标位置 / 期望轨迹 **不在 obs 内**——这是 proprioceptive control 的标准设定。`goal_offset=25` 表示从 history frame 出发经过 25 步后的状态作为 goal frame
- **PushT / TwoRoom**：obs 是 `stable_worldmodel` 返回的 (agent + target + 内部状态) 的拼接，包含 agent 位置/速度与 block/door 等离散几何对象的姿态
- **观测维度差异**：从 cartpole 的 2D 到 humanoid_cmu 的 63D——这是评测“跨 obs_dim 泛化”的核心难度。所有 12 个模型在 G16 训练时 obs 都 0-pad 到 128 维、action 都 0-pad 到 56 维
- **每幅子图的物理读法**：DMC 3D 渲染的角度是 XML 默认视角（前视或侧视）；PushT 是 2D 平面几何，agent 是蓝圆、block 是绿色 T 形（带角度旋转）、target 是红色虚线 T；TwoRoom 是 2D 平面几何，agent 是蓝点、target 是红星、中央墙 + 门洞（door）位于 $x=150$ 附近

下面按子族分组给出 16 个 env 的**详细参数表 + 物理含义**。obs_dim / action_dim 列是 code/core/envs/dmc_env.py:83-100 的 DMC_ENVS 表（或 swm_envs.py:45-170 的对应 wrapper）的注册值；goal_offset 列来自 configs/generalist_16env.json。

F1 经典控制（4 个 DMC + Reacher，共 5 个）：

env	obs	act	goal	每维物理含义	样本观测 (seed=0, 50 步)
cartpole_2d	2	1	25	[cart_x, pole_angle]: 推车水平位置 (米)、摆杆相对直立的角度 (弧度, 0 = 直立)。XML: mujoco/cartpole.xml	[+0.4032, -0.3384]
pendulum_2d	2	1	25	[cos θ , sin θ]: 单摆角度的 (cos θ , sin θ) 展开 (避免角度缠绕); 1D 力矩输入。MuJoCo XML 内部存的是 1 维 θ , wrapper 展开为 2 维	[+0.2967, +0.9550]
finger	3	2	25	[qpos[0..2]]: 双指关节 (前指 proximal / distal 角、目标指轮的旋转角度); 2D action 控制指指尖端 xy 位置	[+0.4220, -0.5861, -0.8207]
ball_in_cup	4	2	25	[cup_x, cup_z, ball_x, ball_z]: 杯的水平/垂直铰链角 (弧度)、小球在支架坐标系下的位置; 目标是把球摇进杯里	[-0.0004, -0.0233, +0.0003, -0.0315]
reacher (4D)	4	2	25	[shoulder_q, elbow_q, target_x, target_y]: 两关节角 (弧度) + 目标位置在 2D 平面坐标 (米, 范围 ± 0.21)。目标在 obs 内但每 episode 重置, 是 sparse POMDP 的代表	[+0.4303, -0.7232, -0.1836, -0.1934]

F2 运动（9 个 DMC）：

env	obs	act	goal	每维物理含义	样本观测 (seed=0, 50 步)
cheetah	9	6	25	[x_pos, z_pos, y_rot, front_hip, front_shin, front_foot, back_hip, back_shin, back_foot]: 2D 半猎豹沿地面奔跑, qpos 含 1 个 root 水平位置 + 1 个 root 高度 + 1 个 root 旋转 + 6 个后腿关节; 6D action = 6 个关节扭矩	[-0.1404, -0.0982, +0.0556, +0.0160, -0.0522, -0.0787, -0.0404, -0.0527, -0.2024]

env	obs	act	goal	每维物理含义	样本观测 (seed=0, 50 步)
walker	9	6	25	[x_pos, z_pos, y_rot, right_hip, right_knee, right_ankle, left_hip, left_knee, left_ankle]: 2D 双足行走器 (MuJoCo walker); 9 个 qpos 含 root 三参数 + 6 个关节	[-0.0743, -0.0050, +0.0590, +0.6904, -1.2262, -0.0472, +0.0987, -0.6197, +0.6887]
hopper	7	4	25	[x_pos, z_pos, y_rot, thigh, leg, foot]: 2D 单腿跳跃器 (root 三参数 + 3 个关节); 4D action 控制 4 个扭矩	[-0.0239, -0.2937, +0.1536, -0.1824, -1.0106, +0.0819, -0.2233]
quadruped	30	12	25	[root_xyz, root_quat(4), ×4 legs (hip×3 + knee×3)/leg]: 3D 四足机器人, 每条腿 3 个关节共 12 个关节; XML qpos 含 root 7 参数 (xyz + 四元数) + 12 关节 = 19; 但 obs_dim=30 因 XML 注册包含额外 root 自由度与脚趾指示变量。12D action = 12 个关节扭矩	[-0.4621, +0.2896, +1.3941, +0.7517, -0.4908, +0.0646, ..., +1.0000, +0.0000, +0.0000, +0.0000]
humanoid	28	21	25	[root_xyz, root_quat(4), 21 关节角]: 2D 简化人形 21 关节 (MuJoCo humanoid.xml, 21 个 actuator 对应 21 个 joint)。注意: XML 注册 obs_dim=28、act_dim=21	[-0.0613, -0.0062, +1.1277, +0.9768, -0.1051, -0.1106, ..., -0.4508, -0.2026, -0.5048, -0.6437]
humanoid_CMU63		56	25	[root_xyz, root_quat(4), cmurange(56)]: 3D 高自由度人形 (CMU mocap 标记); 56 维 cmurange 是 mocap 关节角度, 56 维 action 控制 56 个 actuator	[-0.0004, -0.0002, +0.9503, +0.7367, +0.6759, -0.0203, ..., +0.2063, +0.0861, -0.1953, +0.1751]
dog	87	38	25	[root_xyz, root_quat(4), 38 关节 + 18 速度辅助变量]: 3D 高自由度四足狗 (MuJoCo dog.xml), 38 维 actuator; XML 注册 obs_dim=87 (含额外 root state 与速度估计项)	[+0.0363, +0.0127, +0.1913, +0.9867, +0.1426, +0.0737, ..., +1.0000, -0.0000, +0.0000, +0.0000]
fish	14	5	25	[root_quat(4), 5 关节角, 5 速度, 1 尾巴辅助]: 2D 鱼形游泳 (MuJoCo fish.xml); 5 维 actuator 控制躯体摆动 + 尾部	[+0.0045, -0.0177, +0.1039, +0.9898, +0.0307, -0.1151, ..., -22.0118, +14.5302, +14.8166, +4.2708]

env	obs	act	goal	每维物理含义	样本观测 (seed=0, 50 步)
stacker	20	5	25	[root_xyz, root_quat(4), 5 关节, 5 个 cube 指示变量]: 2D 抓取机械臂堆叠 3 个立方体 (MuJoCo stacker.xml); 5D action 控制末端	[+0.0032, +0.3956, -0.6476, -1.1333, +0.1196, +0.0730, ..., +0.0000, +0.2000, +0.3875, +0.0000]

跨基准族 2 个 2D 几何 env (PushT / TwoRoom):

env	obs	act	goal	每维物理含义	样本观测 (seed=0, 50 步)
pusht	7	2	100	[agent_x, agent_y, agent_vx, agent_vy, block_x, block_y, block_angle]: swm/PushT-v1 的 7D 状态。Agent 是平面圆盘 (半径 12 px), 位置/速度; block 是 60×60 px 的 T 形方块, 位置 + 旋转角度。成功判定: block 位于目标位置 (200,100) ± 70 px 且角度 ± 1.0 rad。action_low/high 来自 gym action_space ($\sim [-1, 1]^2$)	[+163.82, +265.62, +383.77, +318.04, +4.97, -285.17, +347.66]
tworoom	10	2	100	[agent_x, agent_y, target_x, target_y, door?, 5 内部状态]: swm/TwoRoom-v1 的 10D Box 观测。前 4 维是 agent + target 在 2D 平面的位置; 第 5 维是 door 状态 (开放/关闭); 最后 5 维是环境内部状态 (门状态码、agent 是否在房间 A/B、目标房间等)。Goal_offset=100 (典型 LeWM paper 设置); 目标可能在房间 A 或房间 B, agent 不知道——需要工作记忆 / 推断	[+149.17, +84.13, +99.83, +81.18, +112.0, +49.0, +0.0, +0.0, +0.0, +0.0]

为什么 goal_offset 在 DMC/Reach/PushT 之间不同:

- **DMC × 14 + Reacher (goal_offset = 25):** 25 步 $\approx$ 0.5 秒控制循环 (dt=0.02s), agent 短视野即可完成大多数 DMC 任务的“局部姿态调整”; CEM 5 步规划器达不到 25 步目标 (详见 §8.3 Bug #3), 但 mean_cos_dist 仍给出有意义的潜变量对齐信号
- **PushT / TwoRoom (goal_offset = 100):** 100 步 $\approx$ 2 秒控制循环; PushT 需要 agent 绕到 block 后面推动 $\rightarrow$ 必须有长期预测能力; TwoRoom 需要 agent 穿过 door 才能到达另一房间

——单步视野无意义。两者的 LeWM paper 也使用 `100.configs/oodc/cross_benchmark_F4_train_only`。中 `pusht / tworoom` 的 `goal_offset=100` 是 **显式选择**（与 DMC 的 25 形成对比）

每幅子图的展示位置：

- 16-env 总览（图 1）：4×4 网格，每个子图是一个 env 在 `seed=0 / 50` 步后的瞬时渲染
- 单 env 大图（`paper/figs/dataset_samples/<env>.png`, 800×600）：三栏——上：物理场景（DMC 3D 渲染或 2D 几何散点）；中：obs 向量 bar chart（带维度标签）；下：文字 caption（任务描述 + obs_dim + action_dim + 数值化样本）

由 `obs_dim` 看 STJEW 设计的物理负担：从 cartpole 的 2D 到 dog 的 87D，STJEW 通过 G16 训练的 `pad_obs_to=128` 统一接收；每个 env 的样本在进入模型前都先做 per-batch 全局 z-score 归一化，让跨 env 的 obs 在同一尺度——这是跨基准族评测能成立的前提（详见 §5.3“状态标准化”）。

4.8.2 “DMC×14” / “DMC×13” / “+ reacher / tworoom / pusht” 记法说明

本报告（及 `paper.md`、`README.md`）频繁出现“DMC×14”、“DMC×13”、“DMC×12 + reacher”、“DMC×14 + reacher + tworoom”这样的记法。这里**统一说明**以避免混淆：

记法的含义：“DMC×N” = “N 个 DMC 标准 proprioceptive env 的并集”。“DMC×14”是包含 14 个 env 的**满集**；“DMC×13”、“DMC×12”等是**去除某些 env 后的子集**。“+ reacher”、“+ tworoom”、“+ pusht”是在 **DMC 集合外追加**的 1-3 个跨族 env。

“DMC×14”包含哪些 env（按子族分类，共 14 个）：

- **F1 经典控制（4 个）**：cartpole_2d（5 维小车摆杆）、pendulum_2d（4 维单摆）、finger（3 维手指旋转）、ball_in_cup（6 维球进杯）——低维 proprioception + 简单动力学
- **F2 运动（9 个）**：cheetah, walker, hopper, quadruped, humanoid, humanoid_CMU, dog, fish, stacker——9 个，不是 5 个（常有读者误以为“5”）。高维 proprioception + 关节动力学 + 接触
- **F3 稀疏 POMDP（1 个）**：reacher（4D）——2 关节机械臂，target 部分可见
- **合计**：4 + 9 + 1 = 14 个 env。这是“DMC×14”的标准定义

“DMC×13”、“DMC×12”等子集是怎么来的：在 OOD Path-C 的某些 split 中，**训练 env** 是 DMC 的子集（不是全集）。例如：

- **oodc_F1(classic 训练)**：train_envs = cartpole_2d, pendulum_2d, finger, ball_in_cup, cheetah = 5 个 env（“DMC classic (4) + locomotion cheetah (1)”）。评测在 **8 个 held-out env**（walker, humanoid, humanoid_CMU, hopper, quadruped, dog, delayed_t_maze, cheetah_velhidden）上
- **oodc_F1F2 (classic + locomotion 训练)**：train_envs = 10 个 classic + locomotion 全部（缺 cheetah_velhidden 和 reacher）。评测在 **2 个 held-out**（delayed_t_maze, cheetah_velhidden）上

- “DMC×14 + reacher”: 14 个 DMC env + 1 个 reacher = **15 个 env**。跨基准族 F1/F2/F3 都用这个 15-16 env 集
- “DMC×14 + reacher + tworoom”: F1 split 的训练集 = 14 + 1 + 1 = **16 个 env**。评测在 pusht 上（不在训练集）
- “DMC×13” 来自哪里: 跨基准族 F4(DMC held out) 里, 训练用 pusht + tworoom + reacher (3 个非 DMC env), 评测在所有 14 个 DMC env 上。但**为了避免数据泄露**, cheetah_velhidden (一个 stress variant of cheetah) 有时被**有意排除**——这时评测 env 集是 13 个。说 “DMC×13” 几乎总是指 F4 split 的 14 个 DMC env 中**去除 stress variant** 后的子集 (13 个)

为什么 “14” 和 “13” 重要: 评测 cell 数 = models × envs。

- F1/F2/F3 = 12 models × 1 env = 12 cells/split
- F4 (“DMC×14” 版本) = 12 models × 14 envs = 168 cells/split
- F4 (“DMC×13” 版本, 去除 stress variant) = 12 models × 13 envs = 156 cells/split
- **总 cells:** F1 + F2 + F3 + F4(13) = 12 + 12 + 12 + 156 = **192** (本报告使用此版本); F1 + F2 + F3 + F4(14) = 12 + 12 + 12 + 168 = 204

本报告 “192 cells” 的依据: F1 = 12, F2 = 12, F3 = 12, F4 = 156, 12+12+12+156 = 192。这对应 F4 评测 13 个 DMC env (“DMC×13”) 的版本。本报告 v0.7.13 final 的实验采用 “DMC×13” 版 (去除 stress variant cheetah_velhidden), 故 192 cells。

“×12 + reacher” / “×14 + pusht” 等其他变体: 当某个 split 排除了 DMC 集合中的部分 env、但又追加了非 DMC env 时, 用 “×N + X” 表示。例如 “DMC×12 + reacher” = 12 个 DMC env + 1 个 reacher = 13 个 env 训练集 (用于某个 G16 子集 pilot)。这种记法在 paper.md 和 code/scripts/generalist_v0_7_5/ 的注释里出现。

4.9 跨基准族 4 个 held-out family (用于轴 2 评测)

跨基准族评测使用 4 个**范式完全不同的** family, 每个 family 是一个独立的评测单元集:

Family	env_id (实际)	obs_dim	action_dim	goal_off	任务描述
F1 PushT	swm/PushT-v1	7	2	100	2D 块推动: 7D state = (agent pos+vel, block pos+vel+angle+angvel); 成功判定 block 在目标位置 ±0.07 m, 角度 ±1.0 rad
F2 TwoRoom	swm/TwoRoom-v110		2	100	2 房间部分观测导航: 10D Box 观测; 目标位置在 5×5 unit 房间内; 成功判定 dist ≤ 1.0

F3	mujoco/reacher4	2	25	2	关节机械臂到点: 4D	state
Reacher					$= (\mathbf{qpos}[0:2], \mathbf{target}[0:2]);$	
					$\mathbf{max_episode_steps} = 50$	
F4	mujoco/	1-6	25	14	个 proprioceptive DMC 任务 (如	
DMC					§5.1)	

范式差异:

- **PushT**: 接触动力学 + 离散块推动事件; obs 包含 agent 与 block 两组耦合状态
- **TwoRoom**: 纯几何导航 + 部分观测 (agent 不知道目标在哪个房间)
- **Reacher**: 2 关节欠驱动运动学; target 在状态空间内但每个 episode 重置
- **DMC**: 高维 proprioception + 连续控制 + 关节动力学

数据来源路径:

- **PushT**: /home/lx/LeWM/data/pusht_expert_train.h5 (HDF5, env_kind = "pusht")
- **TwoRoom**: /home/lx/LeWM/data/tworoom_extract/tworoom.h5 (HDF5, env_kind = "tworoom")
- **Reacher**: data/dm_control/3d_rollouts_250k/reacher_250k.npz (env_kind = "reacher_4d")
- **DMC**: 14 个 <env>_250k.npz (env_kind = "dmc")

训练集 vs 评测集: 跨基准族训练使用 15-16 个环境的并集 (gating family held out), 评测在 held-out family 上。例如, PushT-held-out 的训练集是 {DMC $\times$ 14, Reacher, TwoRoom} (15 个环境), 评测只在 PushT 上。配置文件:

- configs/oodc/cross_benchmark_F1_train_only.json: PushT held out, 训练用 15 env
- configs/oodc/cross_benchmark_F2_train_only.json: TwoRoom held out, 训练用 15 env
- configs/oodc/cross_benchmark_F3_train_only.json: Reacher held out, 训练用 15 env
- configs/oodc/cross_benchmark_F4_train_only.json: DMC held out, 训练用 3 env

每个文件的 max_windows 字段为 10000, 单 env 上限 10K window。

4.10 数据加载与采样 (窗口构造)

window 是什么: 一个训练/评测单元。一个 window 由两个帧组成: $w = (s_{\text{history}}, s_{\text{goal}})$, 其中 s_{goal} 是从 s_{history} 出发经过 goal_offset 步后的状态。

采样: 从每 env 的 250k 步轨迹中, 以等间距采样生成 max_windows 个 window (默认 10K for 跨基准族、2K for OOD Path-C)。

- **history_size**: 从 window 中取多少"历史帧"作为条件。跨基准族用 history_size=1 (只有当前帧 + 目标帧), 不堆叠历史; 评估 closed-loop 时 closed_loop.py 默认 history_size=3

- **goal_offset**: DMC / Reacher / 多数 env = 25; PushT / TwoRoom = 100。LeWM paper 在 TwoRoom 上用 100——意味着 plan 100 步才能从起点到目标
- **padding 方案**: 当 env 的 obs_dim 不同时, 零 pad 到 pad_obs_to = 128 (G16 设置); action 同样 pad 到 action_dim = 56。模型读 padded obs, 但只取前 action_dim 维送入 env

4.11 状态标准化

全局 z-score vs per-env z-score:

- **per-env**: 每个 env 各自估计均值与方差。优点: 单 env 上训练更稳定; 缺点: 跨 env 时尺度不一致, shared-weight generalist 难以学习
- **全局 (G16 选择)**: 将所有 env 的轨迹合并, 估计一个**联合均值/方差** (通常基于 union dataset 的 0.05 / 0.95 分位)。优点: 跨 env 一致; 缺点: 少数 env 的极端值会拉伸分布

本文采用**全局 z-score**——因为所有 12 模型共享相同的训练集, 且评测时也是同一尺度。这避免了一个隐藏 bug: per-env normalization 在跨基准族评测时会让 z_{history} 与 z_{goal} 在不同尺度上, 导致 cosine 距离偏差。

4.12 STJEW 系列 (6 个读出口径)

所有 STJEW 模型共享**同一个 8.20M 参数的纯 SNN 骨架** (脉冲神经元堆叠 4 层, embed_dim = 192, trainable = 2.70M, total = 8.20M 含冻结的 ViT-Tiny 编码器)。区别**仅在最后一步**——规划器看到什么:

- **trace_only** (默认, 膜电位禁止): $z_{\text{final}} = \text{trace_proj}(r_t)$ 。规划器只能看到有界的、内容感知的轨迹
- **spike_only** (back-compat spike_gated): $z_{\text{final}} = h_t \cdot s_t$ (连续隐状态被脉冲掩码)
- **rate_only**: $z_{\text{final}} = \text{AvgPool1d}(s_t)$ 。纯速率编码
- **no_trace** (ablation): $z_{\text{final}} = h_t$ (ablation, 无轨迹门控)
- **hidden_leak**: $z_{\text{final}} = h_t + \text{trace_proj}(r_t)$ (隐状态 + 轨迹, 混合)
- **membrane_readout** (违反协议): $z_{\text{final}} = h_t$ (planner 可读膜电位, 破坏 membrane-forbidden 协议)
- **raw_spike**: $z_{\text{final}} = \text{Linear}(s_t)$ (纯 spike 投影, v0.7.6 引入, 本实验未启用)

4.12.1 STJEW 架构详解 (MultiComp + MultiCompStack)

MultiCompartmentCell: 单个 SNN 单元, 由 $n_d = 3$ 个 dendrite 室 + 1 个 soma 室组成 (见 code/snn_cell.py)。每条 dendrite 都有自己的时间常数与权重 (自适应), 将输入电流沿树突结构分段整合; soma 在累积电流超过软阈值时发放脉冲。

MultiCompStack: $n_{\text{layers}} = 4$ 层 MultiCompartmentCell 堆叠, 每层后接:

```

s_l = MultiCompCell(h_{l-1})      # 树突脉冲
r_l = post_mlp_l(s_l)             # Linear -> GELU -> Linear
h_l = h_{l-1} + LayerNorm(r_l)    # 残差 + LN

```

post_mlp 是一个 12 倍扩展的 MLP ($\text{hidden} = 12 \times d_{\text{hid}}$), 让单层 SNN 在不增加细胞参数的前提下获得通道混合能力。每层约 0.88M 参数, 4 层共约 3.5M。

forward pipeline (STJEW forward):

```

pixels/state -> encoder -> z_enc (B, T, 192)      # ViT-Tiny frozen + projector
action       -> ActionMLP -> a_emb (B, T, 192)     # 1-layer linear
h_in = z_enc + a_emb
h, spike = MultiCompStack(h_in)                   # 4 layers
trace = GatedSpikeTrace(spike, ctx=[a_emb, h])     # content-aware gated
z_final = readout(h, spike, trace)                 # per ReadoutMode

```

架构参数总结:

- $\text{embed_dim} = 192$: 与 ViT-Tiny hidden_dim 对齐
- $\text{n_layers} = 4$: G16 训练时为减少 compute, 使用 $\text{n_layers} = 2$; spec 阶段为 4
- $\text{n_d} = 3$: 每细胞 3 个 dendrite 室
- $\text{trace_beta} = 0.9$: trace 初始衰减率
- $\text{freeze_encoder} = \text{True}$: ViT-Tiny 不参与训练
- $\text{trainable} = 2.70\text{M}$, $\text{total} = 8.20\text{M}$

4.12.2 Trace dynamics 完整公式

trace $r_t \in [0, 1]^d$ 是内容感知的脉冲后轨迹。其动力学由**两步**定义:

Step 1: 遗忘门 (GatedSpikeTrace.forward):

$$\alpha_t = \sigma(W_{\text{gate}} \cdot [r_{t-1}, s_t, c_t]), \quad W_{\text{gate}} \in \mathbb{R}^{d \times (2d + d_{\text{ctx}})}$$

其中 $c_t = [a_t, h_t] \in \mathbb{R}^{2d}$ 是当前动作嵌入与连续隐藏状态的拼接; W_{gate} 是单层 Linear, bias 初始化使 $\alpha_0 \approx 0.9$ 。

Step 2: trace 更新:

$$r_t = \alpha_t \odot r_{t-1} + (1 - \alpha_t) \odot s_t, \quad r_t \in [0, 1]^d$$

三个内禀性质:

1. **逐维有界于 $[0, 1]$** : 因为 $\alpha_t \in [0, 1]$ 且 $s_t \in \{0, 1\}$, 归纳可得 $r_t \in [0, 1]$
2. **内容感知遗忘**: 当上下文 c_t 表示”输入分布变了”时, α_t 趋近 0——trace 快速遗忘; 反之 α_t 趋近 1——trace 长期保留
3. **事件驱动**: trace 只在 $s_t \neq 0$ 时有显著更新 (无脉冲时 $\alpha < 1$ 导致指数衰减)

4.12.3 6 个 readout 的数学定义 (来自 STJEW._readout)

- **TRACE_ONLY**: $z = \text{trace_proj}(r_t)$, 其中 $\text{trace_proj} = \text{Linear}(d, d)$; honors membrane-forbidden 协议
- **HIDDEN_LEAK**: $z = h_t + \text{trace_proj}(r_t)$ (legacy v2 默认; 部分违反协议, 因为 h_t 暴露了连续值)
- **MEMBRANE_READOUT**: $z = h_t.\text{detach}()$ ——视 h 为离散潜状态; **明确违反协议**
- **SPIKE_GATED** (legacy SPIKE_ONLY): $z = h_t \cdot s_t.\text{detach}()$, 连续 $\times$ 二值掩码; honors 协议
- **RAW_SPIKE**: $z = \text{raw_spike_proj}(s_t.\text{detach}())$; pure spike; honors 协议
- **RATE_ONLY**: $z = \text{AvgPool1d}(s_t.\text{detach}())$, kernel_size=4, stride=1, padding=2; honors 协议
- **NO_TRACE**: $z = h_t$ (无 trace 分支; ablation)

所有 membrane-forbidden 协议的读出口径都满足: z 仅依赖于 $\{s_t, r_t, c_t\}$, **不依赖于 h_t 的连续值**。这是接口合约的数学承诺。

4.13 6 个非 STJEW 对比模型 (详细配置)

- **CuBiFAE baseline** (SNN, trainable = 5.31M, 违反协议): **什么是 CuBiFAE**: 多时间尺度 ALIF (Adaptive Leaky Integrate-and-Fire) SNN。每层 ALIF 维护 v_t , 发放后 refractory 强制重置; ALIF 的 b 自适应阈值随脉冲累积而增加。规划器可读 ALIF 的膜电位 v_t 。配置: d_hid=192, n_layers=4
- **SpikeDreamer baseline** (混合 LIF + Transformer, trainable = 1.58M, 违反协议): 2 层 LIF 脉冲编码后接 2 层 AdaLN-zero Transformer hidden state。规划器可读 Transformer 隐状态。配置: d_snn=128, d_tx=192, num_layers=2, num_heads=8
- **SLT-LIF-MPC-trace** (SNN, trainable = 0.22M, 膜电位禁止): DECOLLE 算法的 LIF 实现, 3 阶段局部损失。读出 $z = \text{moving_average}(s_t)$ 。膜电位禁止接口。配置: d_in=192, embed_dim=192, n_layers=2, trace_beta=0.9, k_avg=4
- **SLT-LIF-MPC-free** (SNN, trainable = 0.26M, 违反协议): 同架构, 但读出 $z = \text{concat}(s_t, v_t)$, 允许规划器读膜电位
- **LeWM Transformer baseline** (Transformer, trainable = 1.58M, 违反协议): LeWM paper 的 2-layer AdaLN-zero Transformer (论文原为 6 层 + 5.07M; G16 用 embed_dim=192, num_layers=2)。规划器可读 Transformer hidden state
- **GRU baseline** (连续循环神经网络, trainable = 7.42M, 违反协议): 3 层 GRU h=576。标准 GRU 隐状态可读

- **MLP baseline** (无状态 FFN, trainable = 1.44M, 作为坍缩对照): hidden_dim=576, num_layers=4, emb_dim=192 纯前馈, 无循环。所有时间步都从当前 obs 重新预测。预期表现为 $\text{div} \rightarrow 0$ 的”坍缩控制”

总 12 个模型: 6 STJEWm 读出口径 + 6 非 STJEWm 对照 (其中 5 个非 MLP 对照都有循环 / 记忆能力; MLP 是无状态对照)。

4.14 4 个 baseline 家族的角色

家族	模型	测试什么
STJEWm 读出口径	trace / spike / rate / no-trace / leak / membrane	同一 SNN 骨架, 6 种接口变量
连续循环对照	LeWm Transformer, GRU, stateless MLP	连续循环、循环、记忆缺失(坍缩对照)
SNN / 神经形态对照	CuBiFAE, SpikeDreamer, SLT-LIF-MPC-trace, SLT-LIF-MPC-free	已有 SNN 世界模型及其各自的轨迹约定

4.14.1 膜电位禁止协议的接口合约

正式定义为:

协议。 给定一个脉冲动力学系统 (v_t, s_t, r_t) 与一个下游规划器, 规划器只能读取 r_t (post-spike trace), 不能读取 v_t (membrane potential)。

协议在接口层强制 (不是训练时正则化项):

- **不是 loss term:** 训练 loss 不显式包含”隐藏 v_t ”的惩罚
- **是 interface contract:** planner.observe() 只接收 r_t ; 即使 v_t 在内部流动, 也不暴露
- **6 个 readout 中 4 个 honor 协议:** trace_only, spike_gated, rate_only, raw_spike
- **2 个 ablation 不 honor 协议:** membrane_readout(直接读 h), hidden_leak($h + \text{trace_proj}(r)$ 仍依赖 h)

4 个 baseline 家族共同构成本文的**可测试声明**: 如果 STJEWm 的”校准非坍缩”性质是膜电位禁止协议的属性, 则 6 个禁止读口的读出口径应全部落在校准区域; 如果是膜电位读出口径的属性, 则仅有 membrane 读出口径应在那里。

4.14.2 为什么 12 模型足够 (可证伪性论证)

核心声明: 膜电位禁止协议 + trace dynamics 自带的 content-aware 衰减 = 跨基准族泛化所需的全部物理偏置。

12 模型的覆盖性:

1. **6 STJEW readout**: 剥离了”哪个接口变量被读到”的影响, 只保留 trace dynamics
2. **4 SNN baseline**: CuBiFAE / SpikeDreamer / SLT-LIF-MPC ($\times 2$) 各自代表一种 SNN 世界模型设计, 提供**对照**——证明 STJEW 不是”SNN 范式本身就行”
3. **2 循环非 SNN baseline**: GRU / LeWM Transformer 代表连续循环 + attention, 证明 STJEW 不是”循环结构本身就行”
4. **1 无状态对照**: MLP 证明 STJEW 不是”前馈结构也行”——它必须 $\text{div} \rightarrow 0$ 坍塌

任何未来提出的新模型都可以归入这 4 家族之一 (或者证明第 5 个家族的存在——这本身就是一个科学贡献)。

5 训练流程

5.1 优化器与学习率调度

所有 12 个模型使用 AdamW (来自 `code/train/train.py:176`), $\text{lr} = 3 \times 10^{-4}$, $\text{weight_decay} = 10^{-3}$ 。AdamW 的默认 $\beta_1 = 0.9$ 、 $\beta_2 = 0.999$ 、 $\epsilon = 10^{-8}$ 在 PyTorch 中是隐式取值, **不做修改**。无学习率调度 (无 cosine decay、无 warmup) ——因为训练 epoch 数极小 (1-5), 调度收益不显著。

梯度裁剪: `torch.nn.utils.clip_grad_norm_(parameters, 1.0)` (`train.py:277`)。clip 范数 = 1.0 防止脉冲稀疏性爆炸时的大梯度。

5.2 损失函数 (3 项联合损失)

STJEW 的训练损失 (`code/native_losses.py:stjewm_loss`) 由 3 项加权组成:

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_{\text{sigreg}} \cdot \mathcal{L}_{\text{sigreg}} + \lambda_{\text{goal}} \cdot \mathcal{L}_{\text{goal}}$$

逐项定义:

- **预测损失** $\mathcal{L}_{\text{pred}} = \text{MSE}(\hat{z}_{t+1:H+T_{\text{pred}}}, \text{sg}(z_{t+1:H+T_{\text{pred}}}))$ —— $T_{\text{pred}} = \min(H, t_{\text{pred}}, \text{goal_offset})$, stop-gradient 阻止坍塌
- **脉冲稀疏正则** $\mathcal{L}_{\text{sigreg}}$ (来自 `code/sigreg.py:SIGReg`): 对 `emb_pre_cell` (最后一层 SNN 之前的 embedding) 应用 17 个固定 knots、1024 个随机方向的 1D 投影 sigmoid, 惩罚每维 spike rate 偏离 $[0.1, 0.9]$ 的目标带——形式上: $\mathcal{L}_{\text{sigreg}} = \mathbb{E}_u [\text{KL}(p_{\text{spike}} \cdot \sigma(u \cdot x) + (1 - p_{\text{spike}}) \cdot \sigma(u \cdot x)^{-1} \parallel \text{Uniform})]$ 让 spike rate 不退化为常数或全 1
- **Goal 预测损失** $\mathcal{L}_{\text{goal}} = \text{MSE}(\hat{z}_{\text{goal}}, \text{sg}(z_{\text{goal}}))$ ——跨越整个 `goal_offset` 步预测目标潜状态; 这是 CEM planner 在 latent 空间搜索时实际优化的目标

具体权重: $\lambda_{\text{sigreg}} = 0.09$ (弱正则, 避免破坏 trace 动态), $\lambda_{\text{goal}} = 0.5$ (goal 是 planner 的实际目标, 给予较大权重)。

不同 baseline 的 native 损失: CuBiFAE 用 `cubifae_loss` (`pred + L1 spike sparsity`); SpikeDreamer 用 `spikedreamer_loss` (`pred + KL + recon + sparse`); SLT-LIF-MPC 用

slt_lif_mpc_loss (pred + sparse + action); 其他基线 (LeWM Transformer / GRU / MLP) 沿用 STJEWB 的 3 项损失。所有损失通过 code/native_losses.NATIVE_LOSS_DISPATCH 在 train.py:219 处按 args.model 分派。

5.3 batch 构造与损失归约

batch 内样本来源: 每个 batch 由 $B = 32$ 个 (history_size 帧 + goal_offset 帧 + 1 帧) 序列组成。在多 env 训练 (G16) 时, B 个样本从所有 env 的混合 buffer 中随机抽取——**不是 per-env batch**, 因此一个 batch 内可能含不同 env 的样本。每个样本的 obs 都已 pad 到 pad_obs_to, 所以模型无需 per-env 区分。

padding 方案: obs_dim pad 到 128 (G16 设置), action_dim pad 到 56。padding 用零填充; 模型读 padded obs; env.step 仅取前 action_dim 维。

损失归约: 所有损失分量在 batch 维度上取 mean(PyTorch F.mse_loss 默认 reduction='mean')。不进行 per-env 归一或加权——因为不同 env 的样本频率已经由 buffer 大小隐式平衡。

5.4 三种训练 regime (完整 breakdown)

总览: 所有训练通过 code/train/train.py 的统一入口; 区别在于传入的 --multi-env-spec JSON 配置和 --epochs:

regime	ckpt 数	epochs	每 env windows	训练入口	用途
单任务 specialist	13 × 14 = 182	1	10K	train.py --env-kind dmc	单 env 评测
通才 G4/G8/G16	12 × 3 = 36	1	8K/16K/32K	train.py --multi-env-spec	跨任务校准性
跨基准族 (F1-F4)	12 × 4 = 48	5	10K per env	--multi-env-spec configs/oodc/cross_benchmark_Fx_train_on	轴 2 评测
OOD Path-C (F1-F3 单, F1F2/F1F3/F2F3 双)	12 × 6 = 72	1	2K	--multi-env-spec configs/oodc/oodc_Fx.json	轴 1 评测

Single-task specialist (13 models × 14 DMC envs = 182 ckpts):

- 每个 ckpt 在单一 DMC env 上训练 1 epoch = 10K 步
- --env-kind dmc --data data/dm_control/<env>_250k.npz
- --epochs 1 --batch 32 --max-windows 10000 --history-size 1 --goal-offset 25
- 输出 ckpt 路径: results/utility/<env>/<model>/seed_0/final.pt

Generalist G4/G8/G16 (12 models $\times$ 3 scales = 36 ckpts):

- G4 = 4 个 env (cartpole_2d, pendulum_2d, cheetah, finger), 8K windows
- G8 = G4 + walker + cheetah_velhidden + pusht + tworoom, 16K windows
- G16 = G8 + cubifae 全部 8 个 env (pusht, reacher, ball_in_cup, tworoom, delayed_t_maze, walker, finger, quadruped), 32K windows
- `--multi-env-spec configs/generalist_Gx_train.json`, 训练 1 epoch
- 输出 ckpt: `results/generalist_Gx/<model>/seed_0/final.pt`

Cross-benchmark F1-F4 (12 models $\times$ 4 splits = 48 ckpts):

- F1 (PushT held out): 训练 DMC $\times$ 14 + reacher + tworoom = 16 env, 5 epochs, 10K windows/env
- F2 (TwoRoom held out): 训练 DMC $\times$ 14 + reacher + pusht = 16 env, 5 epochs
- F3 (Reacher held out): 训练 DMC $\times$ 14 + pusht + tworoom = 16 env, 5 epochs
- F4 (DMC held out): 训练 pusht + tworoom + reacher = 3 env, 5 epochs
- 每个 split 训 12 个 model = $12 \times 4 = 48$ ckpts
- 配置文件: `configs/oodc/cross_benchmark_F<1,2,3,4>_train_only.json` (max_windows=10000)

OOD Path-C F1/F2/F3/F1F2/F1F3/F2F3 (12 models $\times$ 6 splits = 72 ckpts):

- 每个 split 训 12 个 model = $12 \times 6 = 72$ ckpts
- 训练 1 epoch, 每 env max_windows=2000
- 配置文件: `configs/oodc/oodc_F{1,2,3,F1F2,F1F3,F2F3}.json`
- 由 `code/scripts/utility/ood1_path_c.py` 自动调度训练 + 评测

总结: 总 ckpt 数 $182 + 36 + 48 + 72 = 338$ 个, 覆盖 $1008 + 192 = 1200$ 个评测 cell。

5.5 评测-once-eval-everywhere 模式 (OOD Path-C 的特殊 holdout pattern)

为什么 split 持有某个子族: DMC 的 3 个子族 (classic / locomotion / sparse-POMDP) 有不同的动力学本质:

- **classic:** 低维 proprioception + 简单动力学 (cartpole、pendulum、finger、ball_in_cup)
- **locomotion:** 高维 proprioception + 关节动力学 + 接触 (cheetah、walker、hopper、quadruped、humanoid、humanoid_CMU、dog、fish、stacker)
- **sparse-POMDP:** 目标位置在状态外 / 不可观测, 需要推断 (reacher、delayed_t_maze、cheetah_velhidden 等 stress env)

持有其中一个子族 = 测试模型是否在”完全未见过的动力学”上保持校准。这是 working title 的 gating experiment。

eval-once-once-eval-everywhere: 每个 split 训练 1 个 ckpt, 但评测时在所有 14 个 DMC env 上评估 (不仅是 held-out 子族)。这产生两种评测:

- **持有 env** ("held-out"): 测试 OOD 泛化——模型从未见过该 env
- **已见 env** ("seen"): 测试 ID 表现——模型在训练集上

1008 个 cell = 6 splits $\times$ 12 models $\times$ 14 envs 包含了两者。看 **STJEW** 是否对两种都校准: 是 working title 的核心可证伪点。

5.6 计算资源 (GPU、时长、总预算)

Hardware: NVIDIA A100 80GB 或同等 GPU (v0.7.13 实验在 Lambda Labs H100 上完成)。batch_size=32 在 bf16 混合精度下约消耗 4GB 显存/ckpt。

单 ckpt 训练时长:

- **Single-task (1 epoch, 10K windows)**: $\sim$ 5 分钟 (batch=32, 每 epoch 313 步)
- **Cross-bench (5 epochs, $\sim$ 160K windows)**: $\sim$ 25 分钟 (12 模型 $\times$ 16 env 并行 batch)
- **OOD Path-C (1 epoch, $\sim$ 12K windows)**: $\sim$ 3 分钟
- **G16 (1 epoch, 32K windows)**: $\sim$ 30 分钟 (16 env 并行 batch)

评测时长:

- 单 cell (30 episodes $\times$ 1 seed, 100 CEM samples): $\sim$ 30 秒 (单 GPU)
- 总计: 1200 cells $\times$ 30 sec = $\sim$ 10 GPU-hours

总预算: 338 ckpts $\times$ $\sim$ 10 min avg + 1200 cells $\times$ 30 sec $\approx$ 70 GPU-hours。实际实验 (含失败重训、debug) $\sim$ 200 GPU-hours。

6 测试流程

6.1 单 cell 评测流程 (closed-loop)

每个 cell = 在一个 (ckpt, eval_env) 对上做以下步骤:

1. 重置评测环境 (带 seed), 获取初始状态
2. 从数据集采样一个 (history_size 帧, goal_offset 帧后的 goal 帧) 对。不是随机初始化——用真实数据集中的轨迹窗口, 避免目标分布在训练集外
3. 用 ckpt 的 encoder 对 history 帧和 goal 帧分别编码, 得到 z_{history} 和 z_{goal}
4. **CEM 规划**: 以 z_{history} 为初始潜状态, 规划 horizon=5 步的动作序列, 使得预测的潜轨迹到达 z_{goal} 附近。CEM 参数: 100 samples、10 elites、5 iters

5. **滚动执行**规划的第一动作；用真实环境的反馈更新 history；循环直到 horizon 完成

6. **报告**：

- mean_cos_dist: CEM 终止时的潜状态与 z_{goal} 之间的余弦距离
- env-SR: 环境的原生成功判定
- LeWM@0.05: $\Pr[\text{cos}_{\text{dist}} < 0.05]$

什么是 CEM: Cross-Entropy Method 是一种基于采样的规划算法。它每轮在潜状态空间随机采 100 个动作序列，保留最好的 10 个 (elites)，重新估计好的动作分布，再采 100 个。重复 5 轮。最终用最好的动作序列作为规划。CEM 的核心假设是“潜状态空间是好的规划空间”——如果潜状态是坍塌的常数，CEM 就在做无意义的搜索；如果潜状态是过反应的放大器，CEM 的梯度信号爆炸。

6.2 CEM 规划器 (详细算法)

CEM 完整 pseudocode (基于 code/core/cem.py:CEM.plan):

```
function CEM.plan(z_init, z_goal):
    mu = zeros(horizon, action_dim)          # (H, A)
    sigma = ones(horizon, action_dim) * 1.0   # initial std
    for iter in 0..n_iters-1:
        samples = mu + sigma * randn(n_samples, H, A)
        # Action clipping
        samples = clip(samples, action_low, action_high)
        # Score: predict rollout cost per sample
        costs = zeros(n_samples)
        for k in 0..n_samples-1:
            z_roll = rollout(model, z_init, samples[k])  # (H, D)
            costs[k] = cosine_dist(z_roll[-1], z_goal)
        # Select elites
        elite_idx = argsort(costs)[:n_elites]
        elite_samples = samples[elite_idx]
        # Refit distribution
        mu = elite_samples.mean(dim=0)          # (H, A)
        sigma = elite_samples.std(dim=0) + eps  # avoid collapse
    return mu  # (H, A) action sequence
```

默认参数: $n_{\text{samples}}=100, n_{\text{elites}}=10, n_{\text{iters}}=5, \text{horizon}=5, \sigma_{\text{init}} = 1.0$ 。在 code/eval/closed_loop.py 构造 CEM planner。

Receding-horizon 执行: CEM 返回 $H = 5$ 步动作序列 seq; env.step 接受 seq[0]; 之后再次调用 CEM.plan, 传入更新后的 z_{init} 。每 5 步重规划一次 (Receding-horizon control)。

closed_loop 完整 step-by-step (code/eval/closed_loop.py:138-355):

1. 加载 ckpt: 构建模型 (按 ckpt 的 args 字段), 读 final.pt
2. 加载评测数据集: 从 data_path 取 history_size 帧 + goal_offset 帧后的 goal 帧对 (默认 split="in_dist")
3. 对每个 seed (默认 1 seed):
 - (a) 重置 env (带 seed); 若 env 支持 _set_env_state, 从 init_state 设置
 - (b) 编码 history frames (stack 后过 encoder) 得到 z_{history} ; 编码 goal 得到 z_{goal}
 - (c) 循环 eval_budget (默认 50 步): 每步 CEM.plan(z_{init} , z_{goal}) $\rightarrow$ 5 步动作 $\rightarrow$ env.step 5 次; 更新 z_{init}
 - (d) 终止时编码最终状态 z_{final} , 计算 $\cos(z_{\text{final}}, z_{\text{goal}})$
 - (e) 计算 env.check_success (env-native)
4. 聚合所有 seed 的 per-episode 结果: mean_cos_dist、success_rate_lewm、env_SR

6.3 评测 pipeline 中发现并修复的 2 个 bug

在重新评测跨基准族之前,我们发现并修复了 2 个严重的评测 bug(详见 docs/CODE_BUG_AUDIT.md):

1. **Bug #1 (DMC 阈值过松):** 原代码 code/core/envs/dmc_env.py:83-100 的 DMC_ENVS 表对 9 个高维 locomotion env 设置 tol=1.0: cheetah, walker, hopper, quadruped, humanoid, humanoid_cmu, dog, fish, stacker. check_success 计算 $\text{dist} = \|\text{state} - \text{goal}\|_2 / \sqrt{n_q}$, 对 87 维 state 随机 uniform 距离 $\approx 0.45 < 1.0$ ——**随机通过率 87-100%**。这是 DMC 内部子族实验里“SNN 全部 env-SR = 100%”的假象来源。修复: 所有 env tol = 0.1, 随机通过率 0%。修复后 env-SR 全部从 1.0 降到 0 (这是事实, CEM 5 步规划器达不到 25 步目标), 但 mean_cos_dist 显露出真信号: STJEW family 6 个 readout 全部在 [0.094, 0.116], MLP/GRU 接近 0 (坍缩), LeWM-v2 在 0.17-0.19 (过反应)
2. **Bug #2 (LeWM@0.1 阈值被常数 latent 平凡通过):** 原 success_threshold_cos = 0.1 让 MLP (输出常数零) 也能达到 100%。修复: 保留 LeWM@0.05 (v0.7.13 新增, 更严) 和 LeWM@0.01 (最严) 作为 thresholded-success; 同时以原始 mean_cos_dist (raw, threshold-free) 为主指标

Bug 修复的代码 diff (核心行):

```
# code/core/envs/dmc_env.py
"cheetah":      ("cheetah.xml", 9, 6, 200, 0.1), # was 1.0 (random 90% pass)
"humanoid":     ("humanoid.xml", 28, 21, 200, 0.1), # was 1.0 (random 98%)
"dog":          ("dog.xml", 87, 38, 200, 0.1), # was 1.0 (random 100%)
# ... 所有 9 个 env 都从 tol=1.0 改为 tol=0.1

# code/eval/closed_loop.py
success_threshold_cos: float = 0.1, # legacy, kept for back-compat
```

```
# 新增 per-episode record:
lewm_succ_005 = cos_dist < 0.05
lewm_succ_001 = cos_dist < 0.01
# 新增 per-seed aggregate:
"success_rate_lewm_005": float(lewm_005_arr.mean()),
"success_rate_lewm_001": float(lewm_001_arr.mean()),
```

Bug 修复后其他保留的 bug: Bug #3 (CEM horizon=5 vs goal_offset=25/100) **未修复**。horizon=5 是 planner budget 限制, 提升到 25-100 让 CEM planning 计算成本不可行 (5 iters $\times$ 100 samples $\times$ 100 steps)。因此 **env-SR = 0 是 horizon artifact**, 不是模型失败——评测时改用 mean_cos_dist (不依赖 horizon 长度) 作为主指标

6.4 评测 cell 计数 (按 axis 详细 breakdown)

总览:

评测轴	splits $\times$ models $\times$ envs	total cells	目的	配置文件
轴 1 DMC 内部子族	$6 \times 12 \times 14$	1008	gating experiment	configs/oodc/oodc_
轴 2 跨基准族	$4 \times 12 \times \{1-13\}$	192	边界条件	configs/oodc/cross
总计		1200		

轴 1 每个 split 的 cell breakdown:

- oodc_F1: 12 models $\times$ 14 DMC envs (其中 8 个 held-out, 6 个 seen) = 168 cells
- oodc_F2: 12 $\times$ 14 (其中 7 held-out, 7 seen) = 168 cells
- oodc_F3: 12 $\times$ 14 (其中 11 held-out, 3 seen) = 168 cells
- oodc_F1F2: 12 $\times$ 14 (其中 2 held-out, 12 seen) = 168 cells
- oodc_F1F3: 12 $\times$ 14 (其中 6 held-out, 8 seen) = 168 cells
- oodc_F2F3: 12 $\times$ 14 (其中 5 held-out, 9 seen) = 168 cells
- 总计: $6 \times 168 = 1008$ cells

轴 2 每个 split 的 cell breakdown:

- cross_benchmark_F1 (PushT held out): 12 $\times$ 1 = 12 cells
- cross_benchmark_F2 (TwoRoom held out): 12 $\times$ 1 = 12 cells
- cross_benchmark_F3 (Reacher held out): 12 $\times$ 1 = 12 cells
- cross_benchmark_F4 (DMC held out): 12 $\times$ 13 = 156 cells
- 总计: 12 + 12 + 12 + 156 = 192 cells

6.5 评测 pipeline 中发现并修复的 2 个 bug (综述)

详细 bug 修复见 docs/CODE_BUG_AUDIT.md 与本节上文”§8.3”。简要：

1. **Bug #1 (DMC 阈值过松)**: 原 `check_success tol=1.0`, 随机通过率 87-100%。修复 `tol=0.1`, 随机通过率 0%
2. **Bug #2 (LeWM@0.1 阈值被常数 latent 平凡通过)**: 修复后用 `LeWM@0.05` 和原始 `mean_cos_dist` 为主指标

6.6 每个 cell 的 JSON 字段

每个评测 cell 写入一个 JSON 文件(`results/utility/ood1/<split>/<model>/seed_0/<env>.json` 或 `results/cross_benchmark_F<x>/eval/<model>_<env>.json`), 字段定义如下:

- **env_id**: env 的字符串 ID (如 "mujoco/reacher")
- **n_episodes**: 本 cell 跑的 episode 数 (默认 30)
- **n_seeds**: seed 数 (OOD Path-C = 1; 跨基准族 = 1)
- **cem_samples / cem_elites / cem_iters**: CEM 参数
- **horizon**: CEM horizon (默认 5)
- **eval_budget**: 单 episode 最大 env step 数 (默认 50)
- **goal_offset**: goal_offset 步
- **history_size**: 编码时堆叠的帧数 (默认 3)
- **success_rate_lewm**: $\Pr[\cos_dist < 0.1]$ (legacy, kept for back-compat)
- **success_rate_lewm_005**: $\Pr[\cos_dist < 0.05]$
- **success_rate_lewm_001**: $\Pr[\cos_dist < 0.01]$
- **success_rate_env**: env-native 成功判定 (DMC 修复后此字段全 0)
- **mean_cos_dist**: CEM 终止时 z_{final} 与 z_{goal} 的平均余弦距离 (**主指标**)
- **mean_phys_dist**: 物理空间距离 L_2 的均值
- **per_seed**: 每个 seed 的聚合指标列表
- **per_episode**: 每个 episode 的 `cos_dist`、`reward`、`env_success`、`plan_time_sec`
- **wall_time_sec**: 本 cell 的总 wall 时长

物理诊断 cell (仅 OOD Path-C `measure_diagnostic_dmc`) 额外字段:

- **divergence**: per-dim 标准差的均值

- **responsiveness**: $\|\Delta z\|/\|\Delta o\|$ 的均值
- **rho**: Pearson 相关系数
- **per_dim_std_max / min**: 最大 / 最小维的标准差
- **n_steps**: 200 步随机 rollout

7 指标定义

7.1 抗坍缩诊断（先验定义，来自物理约束）

- **divergence-from-constant (div)**: $\sqrt{\text{Var}(z_{t=1..T})}$, 逐维求平均。在 200 步随机策略轨迹上计算。
 - $\text{div} \rightarrow 0$: 坍缩 (MLP 的 $\text{div} \approx 0.0002$)
 - $\text{div} \in [0.008, 0.015]$: 校准
 - $\text{div} > 0.15$: 过反应 (LeWM-v2 的 $\text{div} \approx 0.18$)
- **responsiveness (resp)**: $\frac{1}{T} \sum_{t=1}^T \frac{|\Delta z_t|}{|\Delta o_t|}$ (向量范数)
 - $\text{resp} > 10$: latent 比 obs 更抖 (噪声 / 过反应)
 - $\text{resp} \in [0.15, 0.30]$: 校准
- **event-alignment ρ** : $\text{Pearson}(\Delta o_t, \Delta z_t)$ per step
 - $|\rho| \geq 0.95$: 校准 (STJEW 全部 $\rho \geq 0.99$)
 - $|\rho| < 0.5$: noise / over-reaction

7.2 任务表现指标（事后报告）

- **mean_cos_dist** (主指标): CEM 终止时潜状态与 z_{goal} 的余弦距离。 $\in [0, 2]$, 越小越好。
- **LeWM@0.05**: $\Pr[\text{cos}_{\text{dist}} < 0.05]$, $\in [0, 1]$, 越大越好。
- **env-SR**: 环境的原生成功判定。修复 bug 后所有 $\text{cell} = 0$ (CEM 5 步达不到 25-100 步目标)。
这个指标在修复后失去了信号意义，仅作为 sanity check。

7.3 判定失败的”四象限”

将 4 个家族在 (div, ρ) 平面上分类:

家族 / 模型	div	resp	ρ	失败模式
MLP (无状态对照)	0.0002 ($\times 50$ 太小)	yes	~ 0	坍缩 : 常输出

GRU		0.007 (校准)	yes	-0.07	噪声: latent 跟 obs 不相关
LeWM former	Trans- (×16 太大)	0.186	yes	+0.52	过反应: latent 放大 30×
STJEW readouts)	(6 ±	0.011	yes	≥ 0.99	校准
CuBiFAE		0.001			
		0.012	yes	(under measurement)	校准 (SNN)
SLT-LIF-MPC-trace		0.012	yes	(under measurement)	校准 (SNN)
SLT-LIF-MPC-free		0.010	yes	(under measurement)	校准 (SNN)

关键观察: MLP 的 $\text{div} = 0.0002$ 正是常数 latent 的预期——per-dim 标准差为 0。这不是我们对 GRU 的预期失败模式 ($\text{resp} = 30$ 、 div 正常)，也不是 LeWM 的预期 ($\text{resp} = 30$ 、 $\text{div} \approx 0.19$) ——这两个模型都有正常 div 但把观测变化放大了 30 倍，给出高 LeWM-SR 靠”很响”而不是靠预测。

8 实验结果

8.1 轴 1: DMC 内部子族 (1008 cells)

每个 (split, model) 的 (div , resp , ρ , env-SR) 均值 (节选自 results/utility/ood1_table_v0_7_13.md):

split	family	n	mean div	mean resp	mean ρ	env_SR
classic held-out	SNN-baselines	24	0.0521	0.2107	0.9710	0.8750
classic held-out	STJEW	48	0.0485	0.2102	0.9744	0.8750
classic held-out	non-SNN	24	0.0656	0.9142	0.9082	0.8750
classic+loco held-out	SNN-baselines	6	0.1505	0.2204	0.9908	0.5000
classic+loco held-out	STJEW	12	0.0485	0.2120	0.9922	0.5000

classic+locomotion	SNN	6	0.0612	0.3139	0.9367	0.5000
held-out						
classic+sparse	SNN-baselines	6	0.1505	0.2204	0.9922	0.5000
held-out						
classic+sparse	STJEW	12	0.0685	0.2225	0.9960	0.5000
held-out						
classic+sparse	non-SNN	6	0.0634	1.4299	0.9115	0.5000
held-out						
locomotion	SNN-baselines	6	0.1505	0.2204	0.9908	0.5000
held-out						
locomotion	STJEW	12	0.0444	0.1983	0.9986	0.5000
held-out						
locomotion	non-SNN	6	0.0636	1.9017	0.9236	0.5000
held-out						
locomotion+sparse	SNN-baselines	6	0.1505	0.2204	0.9908	0.5000
held-out						
locomotion+sparse	STJEW	12	0.0154	0.2053	0.9816	0.5000
held-out						
locomotion+sparse	non-SNN	6	0.0640	1.4299	0.9038	0.5000
held-out						
sparse	SNN-baselines	6	0.1505	0.2204	0.9908	0.5000
held-out						
sparse	STJEW	12	0.0660	0.2040	0.9839	0.5000
held-out						
sparse	non-SNN	6	0.0640	1.4299	0.9038	0.5000
held-out						

关键发现: STJEW 6 个读出口径在所有 6 个 split 上都保持 $\rho \geq 0.97$ 和 $\text{resp} \in [0.20, 0.22]$ 。非线性对照家族 (LeWM-v2 的 $\text{resp} \approx 30$) 在抗坍塌诊断上明显偏离。

8.2 轴 2: 跨基准族 (192 cells, 12 模型 $\times$ 4 splits)

每个 (model, split) 的 `mean_cos_dist`(来自 `results/cross_benchmark_F{1,2,3,4}/eval/`):

Model (Family)		F1	F2	F3	F4	Mean
		PushT	TwoRoom	Reacher	DMC	
STJEW 6 read-						
outs						
trace_only	(de-	0.154	0.052	0.100	0.107	0.103
fault)						
rate_only		0.108	0.055	0.087	0.116	0.092

spike_only		0.146	0.058	0.083	0.118	0.101
no_trace	(ablation)	0.113	0.050	0.089	0.130	0.096
hidden_leak		0.171	0.066	0.103	0.125	0.116
mem-brane_readout	(违反)	0.188	0.055	0.121	0.119	0.121
SNN / 神经形态对照						
SLT-LIF-MPC-trace		0.160	0.070	0.078	0.120	0.107
SLT-LIF-MPC-free		0.249	0.062	0.093	0.125	0.127
CuBiFAE baseline		0.310	0.070	0.109	0.108	0.119
非 SNN 对照						
GRU baseline		0.406	0.114	0.001	0.008	0.039 (坍塌 artifact)
MLP baseline		0.155	0.046	0.000	0.001	0.014 (坍塌 artifact)
LeWM	Trans-former	0.365	0.058	0.230	0.225	0.220 (过反应)

关键发现：

1. STJEWM 6 readouts 的 per-model `mean_cos_dist` 在 $[0.091, 0.121]$ ，与轴 1 的校准区 ($\text{div} \in [0.011, 0.013]$ 、 $\rho \geq 0.99$) 一致。
2. 在 4 个 split 中，**STJEWM 6 readouts 都优于 CuBiFAE** (除 F3 上 SLT-LIF-MPC-trace 略胜 STJEWM 0.005)。trace 赢 2 个 (TwoRoom、DMC)，rate 赢 1 个 (PushT)，spike 赢 1 个 (Reacher)。**trace 不是 universal winner**。
3. MLP/GRU 在 F3/F4 上的”低 cos”是**坍塌 artifact**：它们输出常数零 latent，cos 距离自然小，但它们显然不是能规划的世界模型。
4. LeWM-v2 表现最差 (0.220 mean)，**过反应**。

9 为什么 STJEWM 有效 (更好的物理故事)

9.1 两个层次的解释

STJEWM 之所以在 4 个跨基准族 split 上都优于 CuBiFAE，是因为它在**两个正交轴**上同时满足约束：

9.1.1 物理层：trace dynamics 自带校准性

脉冲后的轨迹 (trace) 是一个内容感知的低通滤波器：

$$r_t = \alpha_t \cdot r_{t-1} + (1 - \alpha_t) \cdot s_t \quad \text{其中} \quad \alpha_t = \sigma(W[r_{t-1}, s_t, c_t])$$

几个性质：

- **有界**： $r_t \in [0, 1]$ ，逐维。
- **有遗忘**：没有脉冲时， r 自然衰减 (α 接近 0)，避免无限累积。
- **有更新**：有脉冲时， r 跳到接近 1，记录事件。
- **内容感知**： α 取决于上下文 c_t ，所以”忘记什么、记住什么”由环境调制。

这种 dynamics 在**任何**下游接口下都给出 $\text{div} \approx 0.011$ 、 $\text{resp} \approx 0.21$ 、 $\rho \approx 1$ ——我们在 STJEW 6 个读出口径上都验证了这一点。

9.1.2 任务层：校准的 latent 让规划器能工作

CEM (Cross-Entropy Method) 规划器在 latent 空间搜索动作序列，使得未来 latent 接近目标 latent。规划器能否工作取决于三件事：

1. latent 是否**随观测有意义地变化** ($\text{div} > 0$) ——否则规划器在做”在常数 latent 里搜索”
2. latent 是否**不放大观测** ($\text{resp} \approx 1$) ——否则梯度信号爆炸
3. latent 是否**与观测事件同步** ($\rho \geq 0.95$) ——否则 CEM 的”未来预测”是噪声

STJEW 6 个读出口径全部满足这三点 (物理层)，所以 CEM 规划器在它们之上能工作。MLP / GRU / LeWM-v2 各自在一个点上失败，所以 CEM 在它们之上崩溃。

9.2 为什么 trace / spike / rate 都能赢？

跨基准族的赢家因 split 而异：

- **trace** 赢 F2/F4：在 Tworoom / DMC (稳态连续控制) 上，content-aware 衰减最匹配数据的平稳性
- **rate** 赢 F1：在 PushT (离散事件：块推动、接触) 上，moving average 把离散脉冲压平成平移信号
- **spike** 赢 F3：在 Reacher (稀疏目标) 上，瞬时脉冲直接编码目标位置

6 个读出口径都落在同一校准区 (0.091-0.121)，所以**没有任何一个读出口径的物理优势压倒性地大于其他**。这反而是 STJEW 主张最强的证据：**不是某个特定的 readout magic 赢了，而是 trace dynamics 本身的 calibration property 赢了。**

9.3 膜电位禁止协议的作用

如果取消协议（让规划器读膜电位 h_t ），我们预期：

- STJEW-membrane-readout 应该在 div / ρ 上跟 **trace_only** 一样，因为动力学一样
- 但 CEM 规划器看到的信号是 h_t ，可能给出**不同的**目标距离（因为 h_t 的尺度与 trace 不同）

我们的实验确认了前者（ div / ρ 在 6 个 readout 上都一致），但跨基准族上 membrane_readout 的 mean_cos_dist 略差（0.121 vs trace 的 0.103），因为 h_t 与 goal 处的 h_{goal} 之间的对齐难度高于 trace 与 r_{goal} 。

关键：膜电位禁止协议不改变校准性（那是 trace dynamics 的内禀属性），而是**改变规划器的接口质量**——trace 的有界性 + 内容感知让 CEM 优化更稳定。

10 实验结论

10.1 三个支持性结论

1. **DMC 内部子族校准性转移**（轴 1，1008 cells，gating experiment）：STJEW 6 readouts 在 6 个 split 上都保持 $\rho \geq 0.97$ 、resp $\in [0.20, 0.22]$ 、div $\in [0.011, 0.013]$ ，与 MLP / GRU / LeWM-v2 的坍塌 / 噪声 / 过反应清楚分离。**working title 在 DMC 内部得到支持。**
2. **跨基准族 4 family 任务表现**（轴 2，192 cells）：STJEW 6 readouts 在 4 个 split（PushT / Tworoom / Reacher / DMC）的 mean_cos_dist 上对 CuBiFAE 全面胜出。trace 赢 2 个、rate 赢 1 个、spike 赢 1 个。**特定 readout winner 因 split 而异**，但 STJEW 6 readouts 都赢。**working title 在跨基准族上得到支持。**
3. **规划瓶颈**：所有模型的 env-SR 在 bug 修复后 = 0（CEM 5 步规划器达不到 25-100 步目标）。这是规划器到动作的解码瓶颈，不是潜变量表示的失败。

10.2 两个诚实限制

1. **trace dynamics 自身不具有”普遍硬性能优势”**：6 个 readout 在 mean_cos_dist 上有微小差异（0.05 范围），**不是单一 readout 赢**，是 calibration 共同 property 赢。
2. **跨模态尚未测试**（DMC proprioception vs DMC 像素观测）：需要 raw-obs branch + 像素编码器，是 working title 的下一阶段。

10.3 修正后的工作标题

原 working title——”事件驱动的预测状态动力学是可泛化世界模型的更优归纳偏置”——修正为：

膜电位禁止协议是一种可复用的 SNN 世界模型评测框架；STJEW 是一种可实施的实现，在 DMC 子族内泛化、跨 SNN 家族在内容感知任务上、以及特定持有族（PushT、Tworoom、Reacher、DMC）上具有相对优势，但不具备跨基准族普遍硬性能胜出。

10.4 未来工作

1. 在 Event-Window 任务上做多 seed 训练 (目前 1 seed, 3 seeds eval), 让 STJEWm 胜出的 ~ 2 pp 差距通过显著性检验。
2. 设计"trace-friendly" 任务, 让 trace 自身在 performance 上比 membrane 单独胜出。
3. 加入 raw-obs branch, 让跨基准族实验涵盖像素控制任务 (DMControl pixel-control 等), 验证 working title 的下一阶段。
4. 显式做 inter-cell significance test (per-split 1000 bootstrap), 给出 STJEWm vs CuBiFAE 的 p -value。

11 附录：复现性

11.1 代码仓库

- 主页: <https://github.com/SolarWindRider/STJEWm>
- 训练 ckpt 与原始数据存放于 OBS: obs://lixiang01/STJEWm_NMI/
- 完整实验报告 (详细 per-cell table): 见 OBS 同名路径

11.2 训练与评测超参

- **训练**: AdamW, $lr = 3 \times 10^{-4}$, $weight_decay = 10^{-3}$, $\lambda_{sigreg} = 0.09$, $\lambda_{goal} = 0.5$, $batch = 32$, 1 epoch (DMC 内部子族) / 5 epochs (跨基准族), $embed_dim = 192$, $n_layers = 2$, $n_d = 3$
- **评测**: CEM samples = 100, elites = 10, iters = 5; horizon = 5; 30 episodes / seed; obs_dim pad 到 128, action_dim pad 到 56; DMC 容差 = 0.1

11.3 可复现命令示例

```
# Train all 12 generalist ckpts (16 DMC envs minus walker/humanoid)
python -m code.train.train --multi-env-spec configs/generalist_G16_minus_walker_humanoid.json

# Eval cross-bench F1 (PushT held out)
python -m code.eval.closed_loop --env pusht \
    --ckpt results/generalist_G16_minus_walker_humanoid/stjewm_trace_only/seed_0/final.pt \
    --data /home/lx/LeWM/data/pusht_expert_train.h5 \
    --n-episodes 30 --n-seeds 1 --history-size 3 --goal-offset 25 \
    --eval-budget 50 \
    --out results/cross_benchmark_F1/eval/stjewm_trace_only_pusht.json
```

11.4 术语表 (glossary)

- **CEM**: Cross-Entropy Method, 基于采样的规划算法
- **ckpt**: checkpoint, 训练好的模型参数文件
- **div / resp / ρ** : 三个抗坍缩诊断 (divergence / responsiveness / event-alignment)
- **DMC**: DeepMind Control Suite, 本文的主要 benchmark
- **FFN**: Feed-Forward Network, 前馈网络
- **G4 / G8 / G16**: 通才训练中同时训练 4/8/16 个环境的 ckpt
- **GRU**: Gated Recurrent Unit, 循环神经网络
- **held-out**: 训练集不包含、测试集包含的
- **LeWM**: LeWM paper (参考的先前工作)
- **membrane-forbidden protocol**: 膜电位禁止协议——规划器只能读 trace, 不能读膜电位
- **obs_dim / action_dim**: 观测维度 / 动作维度
- **OOD**: Out-Of-Distribution, 分布外
- **per-cell**: 单个 (ckpt, env) 对
- **ReadoutMode**: trace / spike / rate / no-trace / leak / membrane
- **resp**: responsiveness, 潜状态对观测的放大率
- **SNN**: Spiking Neural Network, 脉冲神经网络
- **split (脉冲)**: 神经元的 0/1 输出
- **STJEW**: Spiking Temporal Joint-Embedding World Model, 本文方法
- **trace**: 脉冲后的轨迹
- **train / eval / split / family / cell / held-out**: 见上下文